PROGRESS REPORT 1
MATA KULIAH PEMROGRAMAN MOBILE

Aplikasi

OdoTrack

(Smart Motorcycle Odometer & Service Tracker)

[Logo Universitas]

Disusun oleh:
Kelompok 6

Firzatunnisa (2411523011)
Loudysa Azisvi Angelia (2411523024)
Nadila Lailany Numai (2411523025)

Dosen Pengampu:
[isi nama dosen]

[Program Studi]
[Fakultas]
[Universitas]
2026

DAFTAR ISI

BAB I Pendahuluan 1

1.1 Latar Belakang 1

1.2 Tujuan Laporan 1

1.3 Ruang Lingkup dan Pembagian Modul 1

BAB II Progress Aplikasi 2

2.1 Status Pengerjaan Fitur dan Layar 2

2.2 Alur Utama Aplikasi 2

2.3 Tampilan Aplikasi 3

2.4 Rencana Pengembangan Berikutnya 4

BAB III Kendala dan Solusi 5

3.1 Merge Conflict pada main.dart 5

3.2 Statistik Bahasa Repository GitHub 5

3.3 API Deprecated pada Flutter Terbaru 6

BAB IV Code Audit dan Refactoring 7

4.1 Gambaran Umum 7

4.2 Contoh 1: Extract Widget ActiveVehicleCard 7

4.3 Contoh 2: Reusable Component QuickActionButton dan ServiceHistoryCard . . . 8

BAB V Koordinasi Tim 11

5.1 Kontribusi Anggota 11

5.2 Strategi Branching Git 11

BAB VI Penutup 13

6.1 Kesimpulan 13

6.2 Rencana Selanjutnya 13

Lampiran 13

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sepeda motor merupakan kendaraan harian yang banyak digunakan mahasiswa, pekerja, dan masyarakat umum. Meski begitu, riwayat servis dan jadwal pajak kendaraan umumnya hanya diingat atau dicatat secara manual sehingga sering terlewat. Servis yang terlambat dapat merusak komponen motor, sedangkan pajak yang terlambat dikenai denda.

Untuk membantu masalah tersebut, kelompok kami membangun OdoTrack, yaitu aplikasi mobile berbasis Flutter yang dapat mencatat data kendaraan, riwayat servis, serta memberi pengingat servis dan pajak dalam satu aplikasi. Kebutuhan aplikasi telah dirumuskan pada Product Requirements Document (PRD) OdoTrack versi 1.0.

1.2 Tujuan Laporan

Laporan ini menyampaikan progress pengerjaan OdoTrack pada tahap Progress Report 1. Isi laporan mengikuti empat aspek penilaian yang ditetapkan dosen, yaitu progress aplikasi, kendala dan solusi, code audit dan refactoring, serta koordinasi tim.

1.3 Ruang Lingkup dan Pembagian Modul

Aplikasi terdiri dari tiga modul dan masing-masing dikerjakan oleh satu anggota seperti pada Tabel 1. Kode program dikelola pada repository GitHub berikut:

https://github.com/firzatunnisa/OdoTrack

Tabel 1. Pembagian modul dan penanggung jawab

Modul	Nama Modul	Penanggung Jawab	Cakupan Pekerjaan
1	Manajemen Kendaraan	Loudysa Azisvi Angelia	Daftar motor, form tambah motor baru, detail dan hapus motor
2	Riwayat Servis	Firzatunnisa	List riwayat servis, form catat servis baru, estimasi servis berikutnya
3	Dashboard dan Pengingat Servis/Pajak	Nadila Lailany Numai	Dashboard, pengingat servis/pajak, Bottom Navigation Bar, desain UI sesuai Figma, integrasi navigasi antar modul

BAB II
PROGRESS APLIKASI

2.1 Status Pengerjaan Fitur dan Layar

Seluruh UI dan navigasi untuk tiga modul sudah selesai 100%. Data yang tampil saat ini masih berupa data contoh (dummy) karena penyimpanan database belum dikerjakan. Rincian status tiap fitur dan layar dapat dilihat pada Tabel 2.

Tabel 2. Status pengerjaan fitur dan layar

No	Modul	Fitur / Layar	Penanggung Jawab	Status	Progress
1	3	Dashboard (kartu kendaraan aktif, ringkasan, aksi cepat)	Nadila	Selesai (UI)	100%
2	3	Pengingat Servis dan Pajak	Nadila	Selesai (UI)	100%
3	3	Bottom Navigation Bar	Nadila	Selesai (UI)	100%
4	3	Desain UI sesuai Figma	Nadila	Selesai (UI)	100%
5	3	Integrasi navigasi antar modul	Nadila	Selesai (UI)	100%
6	1	Daftar Motor	Loudysa Azisvi	Selesai (UI)	100%
7	1	Form Tambah Motor Baru	Loudysa Azisvi	Selesai (UI)	100%
8	1	Detail Motor	Loudysa Azisvi	Selesai (UI)	100%
9	1	Hapus Motor (dialog konfirmasi)	Loudysa Azisvi	Selesai (UI)	100%
10	2	List Riwayat Servis	Firzatunnisa	Selesai (UI)	100%
11	2	Form Catat Servis Baru	Firzatunnisa	Selesai (UI)	100%
12	2	Estimasi Servis Berikutnya	Firzatunnisa	Selesai (UI)	100%
13	Semua	Integrasi database lokal SQLite	Tim	Belum dimulai	0%
14	Semua	Notifikasi lokal	Tim	Belum dimulai	0%

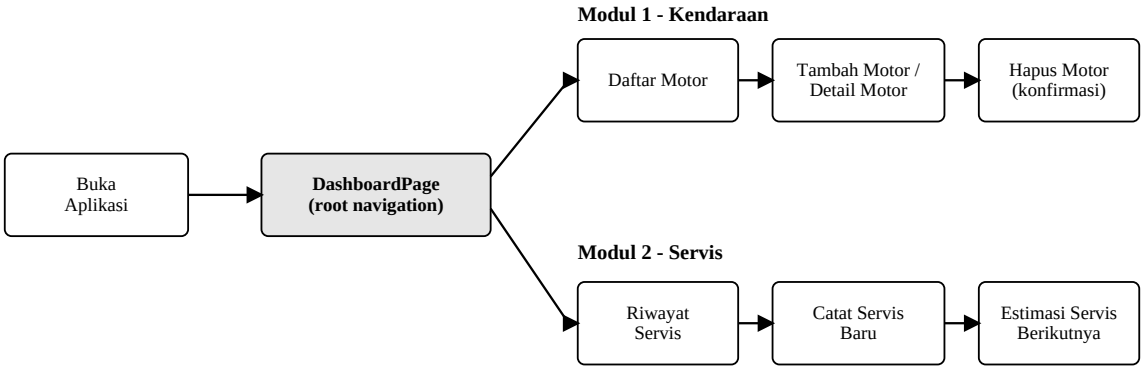
Keterangan: status *Selesai (UI)* berarti tampilan dan navigasi sudah berjalan, sedangkan penyimpanan data permanen akan dikerjakan pada tahap berikutnya.

2.2 Alur Utama Aplikasi

Saat dibuka, aplikasi langsung menampilkan Dashboard (DashboardPage sebagai root navigation). Dari Dashboard terdapat dua alur utama yang sudah berjalan:

1. **Dashboard ke Kendaraan**: pengguna membuka Daftar Motor, lalu dapat menambah motor lewat form, melihat detail motor, dan menghapus motor setelah konfirmasi.
2. **Dashboard ke Servis**: pengguna membuka Riwayat Servis, lalu dapat mencatat servis baru lewat form dan melihat estimasi servis berikutnya.

Perpindahan antar modul dilakukan melalui Bottom Navigation Bar atau tombol aksi cepat di Dashboard. Alur tersebut diringkas pada Gambar 1.



Gambar 1. Alur utama aplikasi OdoTrack

2.3 Tampilan Aplikasi

Berikut tampilan beberapa halaman utama aplikasi.

Tempel screenshot di sini

Gambar 2. Halaman Dashboard

Tempel screenshot di sini

Gambar 3. Halaman List Kendaraan

Tempel screenshot di sini

Gambar 4. Halaman Riwayat Servis

2.4 Rencana Pengembangan Berikutnya

Fitur yang akan dikerjakan berikutnya adalah **integrasi database lokal SQLite** dan **notifikasi lokal**. Selain itu, kami juga merencanakan beberapa pekerjaan pendukung yang ada pada PRD (Tabel 3).

Tabel 3. Rencana pengembangan berikutnya

No	Fitur	Keterangan	Acuan PRD
1	Database lokal SQLite	Menyimpan data kendaraan, riwayat servis, dan pengingat secara permanen menggunakan paket sqflite dalam satu database bersama.	FR-01, FR-05, FR-09, NFR-05
2	Notifikasi lokal	Alert kendaraan tanpa servis baru, servis mendekati jadwal (H-7), dan pengingat bertingkat H-30/H-7/H-1.	FR-04, FR-08, FR-10
3	Data nyata pada UI	Mengganti data dummy dengan data dari SQLite, termasuk perhitungan estimasi servis berikutnya dan ringkasan Dashboard.	FR-06, FR-12
4	Validasi form	Pesan error untuk field wajib kosong, kilometer yang lebih kecil dari servis sebelumnya, dan tanggal jatuh tempo yang sudah lewat.	FR-13, FR-14, FR-15
5	Kamera	Foto kendaraan dan foto nota servis, lengkap dengan izin kamera.	FR-02, FR-07, NFR-07
6	GPS / Maps	Pencarian bengkel dan kantor Samsat terdekat.	FR-11, NFR-07

BAB III
KENDALA DAN SOLUSI

Selama pengerjaan, tim menemui tiga kendala utama. Ringkasannya dapat dilihat pada Tabel 4, kemudian dijelaskan satu per satu.

Tabel 4. Ringkasan kendala dan solusi

No	Kendala	Solusi
1	Merge conflict pada main.dart saat menggabungkan tiga branch anggota.	Membuat branch integrasi ui-integrate dan menjadikan DashboardPage sebagai root navigation.
2	Statistik bahasa GitHub didominasi C++ (35,6%) dan CMake (27,9%).	Mengatur file .gitattributes dengan aturan linguist-vendored sehingga Dart mendominasi (sekitar 85-90%).
3	API deprecated pada Flutter terbaru (withOpacity dan WillPopScope).	Migrasi ke withValues(alpha: ...) dan PopScope.

3.1 Merge Conflict pada main.dart

Setiap anggota mengerjakan modulnya di branch masing-masing dan sama-sama mengubah file main.dart, misalnya untuk halaman awal dan tema. Akibatnya, saat tiga branch digabungkan terjadi konflik pada baris yang sama dan proses penggabungan tertahan.

Untuk mengatasinya, tim membuat branch integrasi khusus bernama **ui-integrate**. Semua branch fitur digabungkan ke branch tersebut secara bertahap dan konflik diselesaikan bersama-sama. Selain itu, **DashboardPage** dijadikan root navigation sehingga main.dart hanya berisi inisialisasi aplikasi dan pemanggilan DashboardPage, sedangkan navigasi antar modul diatur dari Dashboard dan Bottom Navigation Bar. Dengan cara ini, tiga modul berhasil digabungkan dan konflik pada main.dart tidak muncul lagi.

3.2 Statistik Bahasa Repository GitHub

Proyek Flutter membawa folder platform bawaan (windows, linux, macos) yang berisi banyak file C++ dan CMake. Karena itu GitHub menampilkan C++ sebesar 35,6% dan CMake 27,9%, padahal aplikasi ditulis dengan Dart. Hal ini bisa memberi kesan yang keliru saat repository dinilai.

Solusinya adalah menambahkan file **.gitattributes** dengan aturan *linguist-vendored* pada folder bawaan tersebut agar tidak dihitung sebagai kode proyek:

.gitattributes

```
windows/** linguist-vendored
linux/**   linguist-vendored
macos/**   linguist-vendored
```

Setelah konfigurasi ini, Dart mendominasi statistik bahasa repository dengan porsi sekitar 85-90%.

3.3 API Deprecated pada Flutter Terbaru

Pada Flutter versi terbaru, *Color.withOpacity()* dan *WillPopScope* sudah deprecated sehingga muncul peringatan dan berpotensi dihapus di versi mendatang. Kode kami migrasikan ke API penggantinya:

Before

```
color: Colors.black.withOpacity(0.1),

WillPopScope(
  onWillPop: () async => false,
  child: const MyPage(),
)
```

After

```
color: Colors.black.withValues(alpha: 0.1),

PopScope(
  canPop: false,
  child: const MyPage(),
)
```

BAB IV
CODE AUDIT DAN REFACTORING

4.1 Gambaran Umum

Code audit dilakukan dengan meninjau bagian kode yang terlalu panjang (monolitik) dan bagian yang ditulis berulang. Hasilnya dirapikan dengan prinsip modularitas dan DRY (*Don't Repeat Yourself*). Berikut dua contoh refactoring yang sudah dilakukan.

4.2 Contoh 1: Extract Widget ActiveVehicleCard

Masalah. Seluruh kode kartu kendaraan aktif sebelumnya ditulis langsung di dalam *dashboard_page.dart* sehingga file menjadi panjang dan sulit dibaca. **Solusi.** Kartu dipindahkan menjadi widget tersendiri bernama ActiveVehicleCard pada file terpisah.

Before - dashboard_page.dart

```
class DashboardPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ListView(padding: const EdgeInsets.all(16), children: [
        // kartu kendaraan aktif ditulis langsung di sini
        Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(16),
            boxShadow: [BoxShadow(
              color: Colors.black.withValues(alpha: 0.1), blurRadius: 8)],
          ),
          child: Row(children: [
            const Icon(Icons.two_wheeler, size: 48),
            Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
              const Text('Honda Vario 160'),
              const Text('BA 1234 XY'),
              const Text('Odometer: 12.450 km'),
            ]),
          ]),
        ),
        // ... puluhan baris widget lain (pengingat, aksi cepat, dll.)
      ]),
    );
  }
}
```

After - widgets/active_vehicle_card.dart

```
class ActiveVehicleCard extends StatelessWidget {
  final String name;
  final String plate;
  final int odometer;

  const ActiveVehicleCard({
    super.key,
    required this.name,
    required this.plate,
    required this.odometer,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(16),
        boxShadow: [BoxShadow(
          color: Colors.black.withValues(alpha: 0.1), blurRadius: 8)],
      ),
      child: Row(children: [
        const Icon(Icons.two_wheeler, size: 48),
        Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
          Text(name),
          Text(plate),
          Text('Odometer: $odometer km'),
        ]),
      ]),
    );
  }
}
```

After - dashboard_page.dart

```
import '../widgets/active_vehicle_card.dart';

ListView(padding: const EdgeInsets.all(16), children: const [
  ActiveVehicleCard(
    name: 'Honda Vario 160',
    plate: 'BA 1234 XY',
    odometer: 12450,
  ),
  // ... widget lain
]);
```

Dengan perubahan ini, kode Dashboard menjadi lebih ringkas dan mudah dibaca, widget dapat dikembangkan secara terpisah, dan data kartu nantinya mudah diganti dengan data dari SQLite tanpa mengubah tampilan.

4.3 Contoh 2: Reusable Component QuickActionButton dan ServiceHistoryCard

Masalah. Styling tombol aksi cepat di Dashboard dan kartu riwayat servis di modul Servis ditulis berulang dengan kode yang hampir sama, sehingga perubahan desain harus dilakukan di banyak tempat. **Solusi.** Keduanya dijadikan komponen yang dapat dipakai ulang.

Before - styling ditulis berulang

```
// Tombol aksi cepat (diulang untuk setiap aksi)
InkWell(
  onTap: () => Navigator.push(context,
      MaterialPageRoute(builder: (_) => const VehicleListPage())),
  child: Container(
    padding: const EdgeInsets.all(12),
    decoration: BoxDecoration(
      color: const Color(0xFFE8F0FE),
      borderRadius: BorderRadius.circular(12),
    ),
    child: Column(children: [
      const Icon(Icons.two_wheeler), const Text('Kendaraan')]),
  ),
),
InkWell(   // blok yang sama diulang lagi untuk tombol 'Servis'
  onTap: () => Navigator.push(context,
      MaterialPageRoute(builder: (_) => const ServiceHistoryPage())),
  child: Container(/* styling identik */),
),

// Kartu riwayat servis (diulang untuk setiap item)
Container(
  margin: const EdgeInsets.only(bottom: 12),
  padding: const EdgeInsets.all(16),
  decoration: BoxDecoration(
    color: Colors.white,
    borderRadius: BorderRadius.circular(12),
    boxShadow: [BoxShadow(
      color: Colors.black.withValues(alpha: 0.08), blurRadius: 6)],
  ),
  child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
    const Text('Ganti Oli'),
    const Text('10 Agustus 2026 - 12.000 km'),
  ]),
),
```

After - widgets/quick_action_button.dart

```
class QuickActionButton extends StatelessWidget {
  final IconData icon;
  final String label;
  final VoidCallback onTap;

  const QuickActionButton({
    super.key,
    required this.icon,
    required this.label,
    required this.onTap,
  });

  @override
  Widget build(BuildContext context) {
    return InkWell(
      onTap: onTap,
      child: Container(
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          color: const Color(0xFFE8F0FE),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Column(children: [Icon(icon), Text(label)]),
      ),
    );
  }
}
```

After - widgets/service_history_card.dart

```
class ServiceHistoryCard extends StatelessWidget {
  final String title;
  final String date;
  final int odometer;

  const ServiceHistoryCard({
    super.key,
    required this.title,
    required this.date,
    required this.odometer,
  });

  @override
  Widget build(BuildContext context) {
    return Container(
      margin: const EdgeInsets.only(bottom: 12),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [BoxShadow(
          color: Colors.black.withValues(alpha: 0.08), blurRadius: 6)],
      ),
      child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
        Text(title),
        Text('$date - $odometer km'),
      ]),
    );
  }
}
```

After - pemakaian komponen

```
Row(mainAxisAlignment: MainAxisAlignment.spaceEvenly, children: [
  QuickActionButton(
    icon: Icons.two_wheeler,
    label: 'Kendaraan',
    onTap: () => Navigator.push(context,
        MaterialPageRoute(builder: (_) => const VehicleListPage())),
  ),
  QuickActionButton(
    icon: Icons.build,
    label: 'Servis',
    onTap: () => Navigator.push(context,
        MaterialPageRoute(builder: (_) => const ServiceHistoryPage())),
  ),
]),

ServiceHistoryCard(title: 'Ganti Oli', date: '10 Agustus 2026', odometer: 12000),
```

Setelah refactoring, duplikasi kode hilang, tampilan konsisten di seluruh aplikasi, dan perubahan desain cukup dilakukan di satu file. Ringkasan hasil refactoring ada pada Tabel 5.

Tabel 5. Ringkasan hasil refactoring

No	Komponen	Teknik	Dampak
1	ActiveVehicleCard	Extract Widget	dashboard_page.dart lebih ringkas dan modular
2	QuickActionButton	Reusable Component	Duplikasi styling tombol hilang
3	ServiceHistoryCard	Reusable Component	Duplikasi styling kartu hilang dan tampilan konsisten

BAB V
KOORDINASI TIM

5.1 Kontribusi Anggota

Kontribusi masing-masing anggota tim ditunjukkan pada Tabel 6.

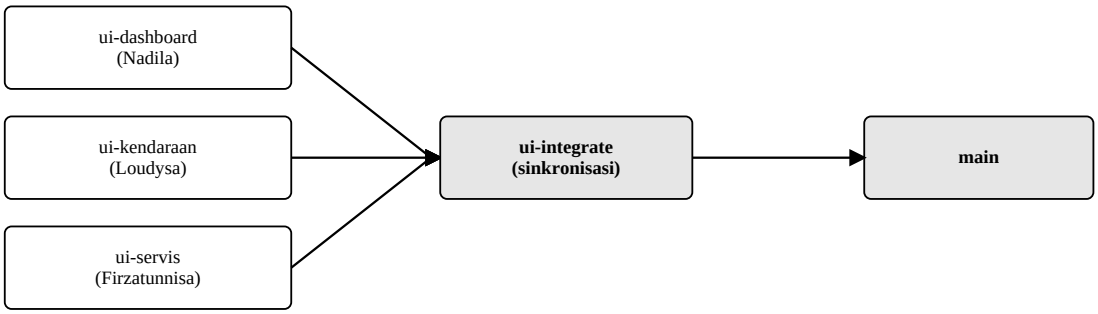
Tabel 6. Kontribusi anggota tim

Nama	Modul	Kontribusi	Branch
Nadila Lailany Numai (2411523025)	Modul 3: Dashboard dan Pengingat Servis/Pajak	Dashboard dan pengingat servis/pajak, Bottom Navigation Bar, desain UI sesuai Figma, integrasi navigasi antar modul.	ui-dashboard
Loudysa Azisvi Angelia (2411523024)	Modul 1: Manajemen Kendaraan	Daftar motor, form tambah motor baru, detail dan hapus motor.	ui-kendaraan
Firzatunnisa (2411523011)	Modul 2: Riwayat Servis	List riwayat servis, form catat servis baru, estimasi servis berikutnya.	ui-servis
Seluruh tim	Pekerjaan bersama	Penyelesaian merge conflict, konfigurasi .gitattributes, migrasi API deprecated, dan refactoring.	ui-integrate

Catatan: jumlah commit tiap anggota dapat dilihat pada riwayat commit di repository GitHub.

5.2 Strategi Branching Git

Tim menggunakan strategi *feature branch*. Setiap anggota bekerja di branch fitur masing-masing supaya pekerjaan tidak saling mengganggu, kemudian semuanya digabungkan di branch **ui-integrate** sebelum masuk ke branch utama (Gambar 5 dan Tabel 7).



Gambar 5. Alur branching Git OdoTrack

Tabel 7. Daftar branch dan fungsinya

Branch	Penanggung Jawab	Fungsi
ui-dashboard	Nadila	Pengembangan UI Dashboard, pengingat, dan Bottom Navigation Bar
ui-kendaraan	Loudysa Azisvi	Pengembangan UI Manajemen Kendaraan
ui-servis	Firzatunnisa	Pengembangan UI Riwayat Servis
ui-integrate	Seluruh tim	Sinkronisasi semua branch fitur, penyelesaian konflik, dan pengujian integrasi
main	Seluruh tim	Branch utama berisi hasil integrasi yang sudah stabil

Langkah kerja yang diterapkan:

1. Setiap anggota mengerjakan fitur di branch masing-masing, lalu melakukan commit dan push ke GitHub.
2. Seluruh branch fitur digabungkan ke ui-integrate dan konflik diselesaikan bersama.
3. Hasil integrasi diuji dengan menjalankan aplikasi, kemudian digabungkan ke main.

BAB VI
PENUTUP

6.1 Kesimpulan

UI dan navigasi untuk tiga modul OdoTrack telah selesai 100% dan terhubung melalui DashboardPage sebagai root navigation. Tiga kendala teknis (merge conflict, statistik bahasa repository, dan API deprecated) telah diatasi. Tim juga telah melakukan refactoring agar kode lebih modular dan tidak berulang, serta menerapkan strategi branching yang terstruktur.

6.2 Rencana Selanjutnya

Tahap berikutnya difokuskan pada integrasi database SQLite dan notifikasi lokal. Setelah itu, tim akan menghubungkan data nyata ke UI, menambahkan validasi form, fitur kamera, dan GPS/Maps sesuai PRD, kemudian melakukan pengujian aplikasi.

LAMPIRAN

1. Repository GitHub: https://github.com/firzatunnisa/OdoTrack
2. Desain Figma: [tempelkan tautan Figma]
3. Dokumen acuan: PRD OdoTrack versi 1.0 (13 September 2026)